AGENTFINANCE AI

F1 · FinTech Onboarding & Intelligence Platform

Technical Architecture · Module Playbook · Database Schema · API Contracts

Hackathon	AIRAVAT 3.0
Problem Statement	F1 — AgentFinance AI
Duration	25 Hours
Stack	Next.js · FastAPI · Supabase · Redis · Whisper · LLaMA 3 · MediaPipe
Modules	10 Independent Modules (M01–M10)
Database	Supabase (PostgreSQL) + Redis
License	All dependencies 100% Open-Source

Module Quick Reference

Module	Name	Tier	Owner	Time
M01	Session & Video Infrastructure	Tier 1	Dev A	2–3 hrs
M02	STT Transcription & Data Extraction	Tier 2	Dev B	3–4 hrs
M03	Consent Capture & Audit Trail	Tier 3	Dev A/B	1–2 hrs
M04	Liveness Detection & Age Estimation	Tier 4	Dev C	3–4 hrs
M05	LLM Intelligence Layer	Tier 5	Dev D	3–4 hrs
M06	Fraud Signal Detection Engine	Tier 6	Dev C/D	3–4 hrs
M07	Mock Bureau & Eligibility Engine	Tier 7	Dev E	2–3 hrs
M08	Loan Offer Generation & Negotiation	Tier 8	Dev D/E	2–3 hrs
M09	Agentic Orchestration & Fallbacks	Tier 9	Dev A/E	3–4 hrs
M10	Compliance Dashboard & Audit Export	Tier 10	Dev B	3–4 hrs

1. Technology Stack — 100% Open Source

Frontend

Library / Tool	Purpose	License	Install
Next.js 14 (App Router)	React framework	MIT	npx create-next-app
TailwindCSS 3	Utility-first CSS	MIT	npm i tailwindcss
shadcn/ui	Component library	MIT	npx shadcn-ui init
Socket.io-client	Real-time events	MIT	npm i socket.io-client
face-api.js	Client-side face detection	MIT	npm i face-api.js
Recharts	Dashboard charts	MIT	npm i recharts

Backend — Core API

Library / Tool	Purpose	License	Install
FastAPI	Python async REST API	MIT	pip install fastapi
Uvicorn	ASGI server	BSD	pip install uvicorn
Socket.io (python)	WebSocket events	MIT	pip install python-socketio
Pydantic v2	Data validation	MIT	pip install pydantic
python-dotenv	Env management	BSD	pip install python-dotenv

Video & Media

Library / Tool	Purpose	License	Install
WebRTC (browser native)	P2P video streaming	W3C Open	Native API
MediaRecorder API	Frame capture	W3C Open	Native API
simple-peer	WebRTC abstraction	MIT	npm i simple-peer

AI / ML — Speech

Library / Tool	Purpose	License	Install
faster-whisper	OpenAI Whisper (CTranslate2)	MIT	pip install faster-whisper
Whisper model: base.en	STT model weights	MIT	Auto-download ~145MB
webrtcvad	Voice activity detection	BSD	pip install webrtcvad

AI / ML — Vision

Library / Tool	Purpose	License	Install
MediaPipe Face Mesh	Liveness + landmark detection	Apache 2.0	pip install mediapipe
DeepFace	Age estimation	MIT	pip install deepface

Library / Tool	Purpose	License	Install
OpenCV	Frame processing	Apache 2.0	pip install opencv-python

AI / ML — LLM

Library / Tool	Purpose	License	Install
Groq API (Llama 3 70B)	Free-tier inference, fast	Llama Community	pip install groq
LangChain	LLM orchestration	MIT	pip install langchain
sentence-transformers	Embeddings	Apache 2.0	pip install sentence-transformers

Data & Storage

Library / Tool	Purpose	License	Install
Supabase (PostgreSQL)	Primary relational database	Apache 2.0	pip install supabase
Redis 7	Session state, pub/sub, cache	BSD	pip install redis
supabase-py	Python Supabase client	MIT	pip install supabase

DevOps & Tooling

Library / Tool	Purpose	License	Install
Docker + docker-compose	Containerization	Apache 2.0	docker compose up
ngrok	Localhost tunneling for demo	Free tier	ngrok http 3000
pytest	Testing	MIT	pip install pytest
ESLint + Prettier	Code quality	MIT	npx eslint --init

2. System Architecture Overview

AgentFinance AI is a **10-tier agentic pipeline** that processes a live video call end-to-end — from session initiation through fraud assessment, credit decisioning, and loan offer delivery. Each module communicates via a well-defined REST or WebSocket contract and emits structured audit events to an immutable append-only log in Supabase. Redis handles all ephemeral session state, pub/sub event bus, and rate-limiting.

Layer	Entry Point	Output	Transport
Browser Client	Tokenised URL	WebRTC stream + WS connection	HTTPS / WSS
M01 Session Gate	Token validation	session_id, device metadata	REST POST
M02 STT Engine	Audio chunk stream	Timestamped transcript JSON	WS stream
M03 Consent Module	Transcript events	Consent record + hash	Internal event
M04 Vision Engine	Video frame JPEG	liveness_score, age_estimate	REST POST
M05 LLM Layer	Full transcript	intent, risk_persona, CoT trace	REST POST
M06 Fraud Engine	Multi-signal inputs	fraud_score 0–100, signal breakdown	REST POST
M07 Bureau API	PAN/applicant_id	credit_score, dpd, loan_count	REST GET
M08 Offer Engine	Bureau + fraud + LLM	personalised loan offer JSON	REST POST
M09 Orchestrator	All module events	session FSM, fallback triggers	Redis pub/sub
M10 Dashboard	Audit log	Compliance report PDF/JSON	REST GET

Data flow summary: Every module reads from and writes to Supabase. Real-time coordination (session heartbeat, stage progression, fallback triggers) travels through Redis Streams. The orchestrator (M09) is the only module allowed to advance the pipeline state machine; all other modules emit events and wait.

Local service port map (docker-compose):

Service	Port	Protocol	Notes
Next.js frontend	3000	HTTP/WS	Entry point for applicants & dashboard
FastAPI — Core API	8000	HTTP/WS	Main backend, all M01–M09 routes
FastAPI — Bureau Mock	8001	HTTP	Isolated mock credit bureau microservice
Redis	6379	Redis protocol	Session state, pub/sub, rate limiting
Supabase Studio	54323	HTTP	Local dev DB GUI
PostgreSQL (Supabase)	54322	TCP	Direct DB access for migrations
faster-whisper server	9000	HTTP	STT inference endpoint (optional sidecar)
ngrok tunnel	Public HTTPS	HTTPS/WSS	Expose :3000 for video-call demo

3. Module Specifications (M01 – M10)

Each module below is **independently deliverable**. Modules communicate only through the defined API contracts — no module imports another's internal code. Colour coding: **Blue** = primary API, **Green** = event emitted to Redis, **Amber** = Supabase write.

M01 Session & Video Call Infrastructure

Tier: Tier 1 | Owner: Dev A | Time: 2–3 hrs

Responsibility: Issue tamper-proof session tokens, establish browser-native WebRTC video sessions, capture device fingerprint and geolocation on session initiation, maintain heartbeat.

Key deliverables:

- Token generation endpoint: `POST /api/sessions/create` — returns signed JWT with session_id
- Token validation middleware — attaches session context to every downstream request
- WebRTC signalling server (Socket.io) for ICE/SDP exchange
- Device fingerprint capture on page load (User-Agent, screen resolution, timezone, canvas hash)
- Geolocation capture via browser Geolocation API (lat/lng stored in session metadata)
- Session heartbeat — Redis key with 5-min TTL, auto-renewed every 30s
- Session state stored in Redis hash `session:{id}`

API Contract:

Method	Endpoint	Input	Output
POST	/api/sessions/create	{ phone, channel }	{ session_id, token, expires_at }
GET	/api/sessions/{id}/validate	Bearer token	{ valid, session_id, stage }
POST	/api/sessions/{id}/metadata	{ device, geo, ip }	{ recorded: true }
WS	/ws/signal/{id}	SDP / ICE events	SDP answer / ICE candidates
DELETE	/api/sessions/{id}	Bearer token	{ terminated: true }

Redis keys written:

- `session:{id}` — HASH — full session metadata, TTL 30 min
- `session:{id}:heartbeat` — STRING — Unix timestamp, TTL 5 min
- `session:{id}:stage` — STRING — current pipeline stage name

M02 STT Transcription & Structured Data Extraction

Tier: Tier 2 | Owner: Dev B | Time: 3–4 hrs

Responsibility: Receive audio chunks from the browser via WebSocket, run faster-whisper (base.en) inference on each chunk, return a timestamped transcript. A secondary LLM pass (LLaMA 3 via Groq) extracts structured application fields with per-field confidence scores.

Key deliverables:

- WebSocket endpoint accepting raw PCM audio (16kHz, mono) or WebM blobs
- Voice Activity Detection with webrtcvad to skip silence chunks (saves ~40% inference calls)
- faster-whisper inference: model=base.en, compute_type=int8 — runs on CPU in <1s per 5s chunk
- Segment accumulation and full transcript assembly in Redis
- LLM extraction prompt: structured JSON output for name, age, employment_type, monthly_income, loan_purpose, requested_amount
- Confidence scoring per field — low-confidence (<0.7) fields flagged for clarification
- Re-extraction triggered on every new transcript segment (streaming update)

API Contract:

Method	Endpoint	Input	Output
WS	/ws/stt/{session_id}	Binary audio chunks	JSON segments with timestamps
GET	/api/transcript/{session_id}	—	{ full_text, segments[], duration_s }
GET	/api/extract/{session_id}	—	{ fields: {name,age,...}, confidence_map }
POST	/api/extract/{session_id}/retry	{ field }	Updated field extraction

LLM Extraction Prompt Structure (Groq / LLaMA 3):

```
System: You are a financial data extraction agent. Extract fields from transcript. Return ONLY valid JSON: {"name":..., "age":..., "employment_type":..., "monthly_income":..., "loan_purpose":..., "requested_amount":..., "confidence":{"name":0.0-1.0, ...}}. If a field is unclear, set value=null and confidence<0.5.
```

M03 Consent Capture & Auditable Consent Trail

Tier: Tier 3 | Owner: Dev A/B | Time: 1–2 hrs

Responsibility: Monitor the live transcript for verbal consent phrases, timestamp and hash the exact consent segment, build a tamper-evident consent record exportable as a signed JSON artifact for regulatory compliance.

Key deliverables:

- Keyword/phrase matcher running on each new transcript segment (regex + fuzzy match)
- Consent phrases: "I agree", "yes I consent", "I accept", "haan mujhe manzoor hai" (Hindi)
- SHA-256 hash of the verbatim transcript segment stored alongside consent record
- Consent record written to Supabase `consent_records` table (immutable — no UPDATE allowed)
- Signed artifact: JSON with session_id, timestamp, phrase, hash, segment_text
- Opt-out detection: "I do not consent" → session flagged, pipeline halted gracefully

API Contract:

Method	Endpoint	Input	Output
POST	/api/consent/{session_id}/check	{ segment_text, timestamp }	{ consent_detected, phrase, hash }
GET	/api/consent/{session_id}	—	{ consent_record, signed_artifact }
GET	/api/consent/{session_id}/export	—	Signed JSON file download

M04 Computer Vision — Liveness Detection & Age Estimation

Tier: Tier 4 | Owner: Dev C | Time: 3–4 hrs

Responsibility: Periodically sample JPEG frames from the WebRTC stream, run liveness detection (anti-spoofing) using MediaPipe Face Mesh, estimate the applicant's age using DeepFace, flag age inconsistency vs. declared age.

Key deliverables:

- Frontend: capture frame every 3s via canvas.toDataURL(), POST to vision endpoint
- MediaPipe Face Mesh: detect 468 landmarks — blink detection via EAR (Eye Aspect Ratio) proves liveness
- Anti-spoofing heuristics: check for texture variance (photo has low variance), blink rate, head pose jitter
- DeepFace age estimation: AgeGenderModel (lightweight, no GPU needed)
- Age delta check: |estimated_age - declared_age| > threshold (default: 8 years) → flag session
- Liveness score (0–1) stored per frame, rolling average over last 5 frames
- Session flagged if liveness_avg < 0.6 for > 15 consecutive seconds
- All frame analysis results logged to Supabase `vision_events`

API Contract:

Method	Endpoint	Input	Output
POST	/api/vision/analyze	{ session_id, frame_b64, timestamp }	{ liveness_score, age_estimate, flags }
GET	/api/vision/{session_id}/summary	—	{ avg_liveness, age_avg, frame_count, fla

M05 LLM Intelligence Layer — Intent & Risk Persona

Tier: Tier 5 | Owner: Dev D | Time: 3–4 hrs

Responsibility: Process the full session transcript through a structured LLM reasoning chain to assign a loan intent category, a risk persona label, and produce a chain-of-thought (CoT) reasoning trace. No black-box outputs — every classification must be explainable.

Key deliverables:

- Intent classification: home_improvement, medical, education, debt_consolidation, business, personal, wedding, travel
- Risk persona: first_time_borrower, conservative_borrower, distressed_borrower, high_income_stable, self_employed_variable
- Structured CoT trace: {"reasoning_steps": [...], "evidence": [...], "conclusion": "..."}
- Tone analysis: detect financial distress signals (desperation language, urgency markers)
- Contradiction detection: cross-check stated income vs. loan amount vs. purpose plausibility
- Multi-turn context: include previous clarification Q&A; in the LLM context
- All outputs immutably logged to `llm_assessments` table with full prompt

API Contract:

Method	Endpoint	Input	Output
POST	/api/llm/analyze	{ session_id, transcript, extracted_fields }	{ intent, risk_persona, cot_trace, flags }

Method	Endpoint	Input	Output
POST	/api/llm/clarify	{ session_id, field, context }	{ question_text } (for TTS)
GET	/api/llm/{session_id}/assessment	—	Full LLM assessment record

M06 Multi-Signal Fraud Detection Engine

Tier: Tier 6 | Owner: Dev C/D | Time: 3–4 hrs

Responsibility: Compute a composite fraud risk score (0–100) from at least five independent algorithmic signals. LLM-only detection is explicitly rejected by the PS — each signal must be computed deterministically.

Signal	Method	Weight	High-risk threshold
Geolocation mismatch	IP geolocation vs. stated city (MaxMind GeoLite2 — free)	20%	Distance > 200km
Device anomaly	Detect emulator (no WebGL), VPN (AS number check), headless browser	20%	Any flag = max score
Age inconsistency	M04 estimated age vs. declared age	15%	|delta| > 8 years
Transcript contradiction	LLM contradiction flag from M05 + regex income patterns	25%	Contradiction detected
Velocity check	Redis: same device/IP applied in last 24h (sliding window)	20%	>1 application

Score interpretation: 0–30 = Low Risk (pass), 31–60 = Medium (review), 61–100 = High (flag/reject)

API Contract:

Method	Endpoint	Input	Output
POST	/api/fraud/score	{ session_id, signals_bundle }	{ fraud_score, breakdown, risk_level }
GET	/api/fraud/{session_id}	—	Full fraud report with per-signal scores

M07 Mock Bureau & Policy-Based Eligibility Engine

Tier: Tier 7 | Owner: Dev E | Time: 2–3 hrs

Responsibility: Build and run a mock credit bureau as a live HTTP microservice (port 8001). Implement a configurable policy rules engine that evaluates eligibility gates and produces a machine-readable decision with full audit trail.

Mock Bureau Response schema:

```
{ "applicant_id": "...", "credit_score": 720, "active_loan_count": 2, "dpd_history": [{"days": 30, "count": 1}], "written_off_accounts": 0, "inquiry_count_90d": 3, "foir": 0.42 }
```

Eligibility rules (configurable via JSON config file):

Rule ID	Field	Operator	Threshold	Action on fail
R01	credit_score	>=	650	REJECT
R02	written_off_accounts	==	0	REJECT
R03	foir (EMI-to-income ratio)	<=	0.50	REJECT

Rule ID	Field	Operator	Threshold	Action on fail
R04	dpd_history last 12 months	==	None	REFER (manual review)
R05	active_loan_count	<=	4	REFER
R06	fraud_score (M06)	<=	60	REJECT if >60
R07	liveness_avg (M04)	>=	0.6	REJECT if <0.6
R08	consent_detected (M03)	==	true	HALT (no consent)

Decision output: **PASS** → forward to M08, **REFER** → human review queue, **REJECT** → end session with reason

M08 Loan Offer Generation & In-Call Negotiation Agent

Tier: Tier 8 | Owner: Dev D/E | Time: 2–3 hrs

Responsibility: Generate a personalised loan offer based on bureau + fraud + LLM outputs. An LLM negotiation sub-agent handles objections in real time, simulates EMI impact, and logs every accepted/rejected offer term.

Offer formula:

- **Eligible amount:** min(requested_amount, 0.3 × annual_income, bureau_limit)
- **Interest rate:** base_rate (11%) + risk_premium (0–4% based on credit_score band) + fraud_premium (0–2%)
- **Tenure options:** 12, 24, 36, 48, 60 months — presented with EMI for each
- **EMI formula:** P × r × (1+r)^n / ((1+r)^n - 1) where r = monthly rate
- **Negotiation rounds:** up to 3 counter-offers allowed before final decision

API Contract:

Method	Endpoint	Input	Output
POST	/api/offer/generate	{ session_id }	{ offer_id, amount, rate, tenure_options, emi
POST	/api/offer/{offer_id}/negotiate	{ counter_request, round_no }	{ revised_offer or final_rejection }
POST	/api/offer/{offer_id}/accept	{ session_id, tenure_chosen }	{ confirmation, final_terms }

M09 Agentic Orchestrator & Session State Machine

Tier: Tier 9 | Owner: Dev A/E | Time: 3–4 hrs

Responsibility: Manage the pipeline as a finite-state machine (FSM). Every stage transition is validated, logged, and recoverable. Implements at least 5 named failure modes with graceful degradation paths.

Pipeline FSM stages:

Stage	Trigger	Success → Next	Failure → Action
SESSION_INIT	Token click	VIDEO_CONNECTED	TOKEN_EXPIRED: issue new link
VIDEO_CONNECTED	WebRTC connected	STT_ACTIVE	VIDEO_FAIL: fallback to audio-only
STT_ACTIVE	First transcript	CONSENT_PENDING	STT_FAIL: retry 2x, then text input

Stage	Trigger	Success → Next	Failure → Action
CONSENT_PENDING	Consent phrase detected	LIVENESS_CHECK	NO_CONSENT: halt gracefully
LIVENESS_CHECK	Vision analysis done	FIELD_EXTRACTION	LIVENESS_FAIL: reject session
FIELD_EXTRACTION	All fields extracted	LLM_ANALYSIS	LOW_CONFIDENCE: re-ask question
LLM_ANALYSIS	Intent + persona done	FRAUD_SCORING	LLM_TIMEOUT: fallback prompt
FRAUD_SCORING	Score computed	BUREAU_CHECK	FRAUD_HIGH: flag + refer
BUREAU_CHECK	Bureau response	OFFER_GENERATION	BUREAU_TIMEOUT: use cached mock
OFFER_GENERATION	Offer created	NEGOTIATION	INELIGIBLE: reject with reason
NEGOTIATION	Offer accepted	SESSION_COMPLETE	MAX_ROUNDS: present best offer
SESSION_COMPLETE	Offer accepted	— END —	Audit log finalised

Named failure modes (Redis key: `session:{id}:failures` — LIST):

- **FM-01 STT_FAILURE:** faster-whisper timeout → fallback to text input form overlay
- **FM-02 LIVENESS_FAILURE:** avg_liveness < 0.6 → 1 retry with better lighting prompt, then reject
- **FM-03 BUREAU_TIMEOUT:** mock bureau >5s → use last cached score, mark as REFER
- **FM-04 VIDEO_DISCONNECT:** WebRTC ICE failure → offer resume link valid 10 min, restore FSM state
- **FM-05 FRAUD_ESCALATION:** fraud_score > 80 → auto-terminate, emit compliance alert
- **FM-06 LLM_HALLUCINATION:** confidence < 0.5 on critical fields → re-ask targeted question

---

M10 Compliance Dashboard & Audit Export

Tier: Tier 10 | Owner: Dev B | Time: 3–4 hrs

Responsibility: Back-office web dashboard (Next.js) displaying all sessions, consent trail viewer, fraud flag summary, offer analytics, and pipeline drop-off funnel. Generate machine-readable regulatory audit reports from live session data.

Dashboard views:

- **Sessions list:** paginated table with status badge, consent check, fraud score, offer outcome
- **Session detail:** full transcript, per-stage timeline, liveness score chart, consent artifact viewer
- **Fraud summary:** histogram of fraud scores, top flag types, flagged session list
- **Funnel analytics:** stage-by-stage drop-off rate (SESSION_INIT → SESSION_COMPLETE)
- **Offer analytics:** acceptance rate, avg approved amount, avg interest rate, negotiation round distribution

Audit report (one-click per session):

```
{ "report_version": "1.0", "generated_at": "ISO-8601", "session_id": "...",
"video_session_ref": "...", "transcript_hash": "sha256:...", "consent_timestamp": "...",
"consent_phrase": "...", "liveness_avg": 0.87, "age_delta": 2, "fraud_score": 24,
"fraud_signals": {...}, "bureau_decision": "PASS", "bureau_credit_score": 720,
"offer_accepted": true, "final_terms": {...}, "pipeline_stages": [...], "failure_events": [] }
```

4. Complete Database Schema

All tables live in Supabase (PostgreSQL 15). Row-Level Security (RLS) is enabled on all tables. Audit tables are append-only — no UPDATE or DELETE allowed (enforced via RLS policy + triggers). Redis is used exclusively for ephemeral session state — nothing in Redis is the source of truth.

Table: `sessions`

Column	Type	Constraints	Description
id	UUID	PK, DEFAULT gen_random_uuid()	Primary session identifier
token_hash	TEXT	NOT NULL, UNIQUE	SHA-256 of issued JWT token
phone	TEXT	NOT NULL	Applicant phone (masked after session)
channel	TEXT	DEFAULT 'sms'	Delivery channel: sms | whatsapp | email
status	TEXT	NOT NULL, DEFAULT 'initiated'	FSM stage name
ip_address	INET		Session originating IP
geo_lat	DECIMAL(9,6)		Browser geolocation latitude
geo_lng	DECIMAL(9,6)		Browser geolocation longitude
device_fingerprint	JSONB		UA, screen, timezone, canvas hash
device_flags	JSONB	DEFAULT '{}'	VPN, emulator, headless flags
created_at	TIMESTAMPTZ	NOT NULL, DEFAULT now()	
updated_at	TIMESTAMPTZ	NOT NULL, DEFAULT now()	Updated by trigger
expires_at	TIMESTAMPTZ	NOT NULL	Session expiry (30 min from create)
completed_at	TIMESTAMPTZ		Set when SESSION_COMPLETE

Table: `transcripts`

Column	Type	Constraints	Description
id	UUID	PK	
session_id	UUID	FK → sessions.id, NOT NULL	
segment_index	INTEGER	NOT NULL	Ordered chunk index
text	TEXT	NOT NULL	Transcribed text for this segment
start_ms	INTEGER	NOT NULL	Offset from session start (milliseconds)
end_ms	INTEGER	NOT NULL	
confidence	DECIMAL(4,3)		Whisper word-level avg confidence 0–1
speaker	TEXT	DEFAULT 'applicant'	applicant | system
created_at	TIMESTAMPTZ	NOT NULL, DEFAULT now()	

Table: `extracted_fields`

Column	Type	Constraints	Description
id	UUID	PK	
session_id	UUID	FK → sessions.id	
version	INTEGER	NOT NULL, DEFAULT 1	Incremented on each re-extraction
name	TEXT		Extracted full name
age	INTEGER		Extracted age
employment_type	TEXT		salaried | self_employed | business | student
monthly_income	DECIMAL(12,2)		In INR
loan_purpose	TEXT		
requested_amount	DECIMAL(12,2)		In INR
confidence_map	JSONB	NOT NULL	{"name": 0.92, "age": 0.85, ...}
low_confidence_fields	TEXT[]		Fields with confidence < 0.7
created_at	TIMESTAMPTZ	NOT NULL, DEFAULT now()	

Table: `consent_records` (append-only)

Column	Type	Constraints	Description
id	UUID	PK	
session_id	UUID	FK → sessions.id, NOT NULL	
consent_type	TEXT	NOT NULL	verbal_explicit | verbal_implicit | opt_out
phrase_detected	TEXT	NOT NULL	Exact phrase that triggered detection
segment_text	TEXT	NOT NULL	Full transcript segment containing consent
segment_hash	TEXT	NOT NULL	SHA-256(segment_text) — tamper evidence
timestamp_ms	BIGINT	NOT NULL	Milliseconds from session start
detected_at	TIMESTAMPTZ	NOT NULL, DEFAULT now()	

Table: `vision_events`

Column	Type	Constraints	Description
id	UUID	PK	
session_id	UUID	FK → sessions.id	
frame_index	INTEGER	NOT NULL	Sequential frame number
captured_at_ms	BIGINT	NOT NULL	Offset from session start
liveness_score	DECIMAL(4,3)	NOT NULL	0.0 – 1.0
age_estimate	INTEGER		DeepFace estimated age

Column	Type	Constraints	Description
blink_detected	BOOLEAN	DEFAULT false	EAR threshold crossed
head_pose	JSONB		{"yaw": float, "pitch": float, "roll": float}
flags	TEXT[]		Anomaly flags: static_image | low_variance | no_face
model_used	TEXT	NOT NULL	mediapipe_face_mesh | deepface_age
created_at	TIMESTAMPTZ	NOT NULL, DEFAULT now()	

Table: `llm_assessments`

Column	Type	Constraints	Description
id	UUID	PK	
session_id	UUID	FK → sessions.id	
assessment_version	INTEGER	DEFAULT 1	Re-runs trigger new version
intent_category	TEXT	NOT NULL	home_improvement | medical | education | ...
intent_confidence	DECIMAL(4,3)	NOT NULL	
risk_persona	TEXT	NOT NULL	first_time | conservative | distressed | ...
persona_confidence	DECIMAL(4,3)	NOT NULL	
cot_trace	JSONB	NOT NULL	Full chain-of-thought reasoning steps
tone_flags	TEXT[]		distress_signals | urgency_markers | inconsistency
contradiction_detected	BOOLEAN	DEFAULT false	
contradiction_detail	TEXT		Human-readable contradiction description
full_prompt	TEXT	NOT NULL	Exact prompt sent to LLM (for audit)
model_used	TEXT	NOT NULL	e.g. llama3-70b-8192 via groq
tokens_used	INTEGER		
latency_ms	INTEGER		
created_at	TIMESTAMPTZ	NOT NULL, DEFAULT now()	

Table: `fraud_reports`

Column	Type	Constraints	Description
id	UUID	PK	
session_id	UUID	FK → sessions.id, UNIQUE	One report per session
fraud_score	INTEGER	NOT NULL, CHECK(0-100)	Composite score
risk_level	TEXT	NOT NULL	low | medium | high
geo_mismatch_score	INTEGER	NOT NULL	Sub-score 0–20

Column	Type	Constraints	Description
device_anomaly_score	INTEGER	NOT NULL	Sub-score 0–20
age_inconsistency_score	INTEGER	NOT NULL	Sub-score 0–15
transcript_contradiction_score	INTEGER	NOT NULL	Sub-score 0–25
velocity_score	INTEGER	NOT NULL	Sub-score 0–20
signal_breakdown	JSONB	NOT NULL	Full breakdown with evidence per signal
created_at	TIMESTAMPTZ	NOT NULL, DEFAULT now()	

Table: `bureau_responses`

Column	Type	Constraints	Description
id	UUID	PK	
session_id	UUID	FK → sessions.id	
applicant_id	TEXT	NOT NULL	PAN hash (never store raw PAN)
credit_score	INTEGER	NOT NULL	Simulated CIBIL score 300–900
active_loan_count	INTEGER	NOT NULL	
dpd_history	JSONB	NOT NULL	[{"days": 30, "count": 1, "year": 2023}]
written_off_accounts	INTEGER	NOT NULL, DEFAULT 0	
inquiry_count_90d	INTEGER	NOT NULL	Hard inquiries in last 90 days
foir	DECIMAL(5,4)	NOT NULL	Fixed Obligation to Income Ratio
eligibility_decision	TEXT	NOT NULL	PASS | REFER | REJECT
rules_evaluated	JSONB	NOT NULL	[{"rule_id": "R01", "passed": true, "value": 720}]
response_source	TEXT	DEFAULT 'mock'	mock | cached | live (future)
fetched_at	TIMESTAMPTZ	NOT NULL, DEFAULT now()	

Table: `loan_offers`

Column	Type	Constraints	Description
id	UUID	PK	
session_id	UUID	FK → sessions.id	
offer_version	INTEGER	DEFAULT 1	Incremented on each counter-offer
principal_amount	DECIMAL(12,2)	NOT NULL	Approved loan amount INR
interest_rate_pct	DECIMAL(5,2)	NOT NULL	Annual rate (e.g. 13.50)
tenure_months	INTEGER		Chosen tenure (null until accepted)
emi_amount	DECIMAL(10,2)		Monthly EMI (computed on tenure choice)

Column	Type	Constraints	Description
emi_table	JSONB	NOT NULL	[{"tenure": 12, "emi": 9200}, ...]
negotiation_round	INTEGER	NOT NULL, DEFAULT 0	
status	TEXT	NOT NULL	pending | accepted | rejected | countered
accepted_at	TIMESTAMPTZ		
created_at	TIMESTAMPTZ	NOT NULL, DEFAULT now()	

Table: `audit_events` (immutable event log — append only)

Column	Type	Constraints	Description
id	UUID	PK	
session_id	UUID	FK → sessions.id, NOT NULL	
event_type	TEXT	NOT NULL	stage_transition | failure | fraud_flag | offer_event | consent |
from_stage	TEXT		Previous FSM stage
to_stage	TEXT		New FSM stage
module	TEXT	NOT NULL	M01 – M10
payload	JSONB	NOT NULL	Full event payload (module-specific)
actor	TEXT	DEFAULT 'system'	system | applicant | operator
created_at	TIMESTAMPTZ	NOT NULL, DEFAULT now()	Immutable timestamp

Redis Key Schema

Key Pattern	Type	TTL	Description
session:{id}	HASH	30 min	Full session metadata: status, stage, device, geo
session:{id}:heartbeat	STRING	5 min	Unix timestamp of last heartbeat
session:{id}:stage	STRING	30 min	Current FSM stage name
session:{id}:transcript	STRING	30 min	Accumulated full transcript text
session:{id}:failures	LIST	30 min	Ordered list of failure mode events
session:{id}:liveness	LIST	30 min	Last 10 liveness scores (sliding window)
session:{id}:fraud_signals	HASH	30 min	Per-signal fraud scores (pre-composite)
velocity:{device_hash}	SORTED SET	24 hrs	Score=timestamp, member=session_id
velocity:{ip}	SORTED SET	24 hrs	IP-level velocity tracking
rate_limit:{ip}	STRING	1 min	Request count for rate limiting
pipeline:events	STREAM	Permanent	Redis Stream: all audit events from all modules
bureau:cache:{applicant_id}	STRING (JSON)	10 min	Cached bureau response (stale fallback)

Key Pattern	Type	TTL	Description
sessions:active	SET	No TTL	Set of all currently active session IDs

5. Inter-Module Contracts & Event Bus

All modules communicate through two channels: **(1) REST HTTP** for synchronous request-response, and **(2) Redis Stream** `pipeline:events` for asynchronous event-driven coordination. M09 (Orchestrator) is the sole consumer that advances the FSM on receipt of events.

Redis Stream event envelope:

```
{ "event_id": "uuid", "session_id": "uuid", "module": "M02", "event_type": "STT_SEGMENT_READY",
"timestamp": "ISO-8601", "payload": { ...module-specific data... } }
```

Event Type	Producer	Consumed By	Triggers FSM transition
SESSION_CREATED	M01	M09	SESSION_INIT → VIDEO_CONNECTED
VIDEO_READY	M01 (WS)	M09	VIDEO_CONNECTED → STT_ACTIVE
STT_SEGMENT_READY	M02	M03, M09	Consent check + field update
TRANSCRIPT_COMPLETE	M02	M05, M09	STT_ACTIVE → LLM_ANALYSIS
CONSENT_DETECTED	M03	M09	CONSENT_PENDING → LIVENESS_CHECK
LIVENESS_PASS	M04	M09	LIVENESS_CHECK → FIELD_EXTRACTION
LIVENESS_FAIL	M04	M09	Triggers FM-02
FIELDS_EXTRACTED	M02	M09	FIELD_EXTRACTION → LLM_ANALYSIS
LLM_ASSESSMENT_DONE	M05	M06, M09	LLM_ANALYSIS → FRAUD_SCORING
FRAUD_SCORED	M06	M07, M09	FRAUD_SCORING → BUREAU_CHECK
BUREAU_RESPONSE	M07	M08, M09	BUREAU_CHECK → OFFER_GENERATION
OFFER_GENERATED	M08	M09	OFFER_GENERATION → NEGOTIATION
OFFER_ACCEPTED	M08	M09, M10	NEGOTIATION → SESSION_COMPLETE
FAILURE_EVENT	Any module	M09	Triggers named failure mode handler

6. Project Directory Structure

```
agentfinance-ai/ ■■■ frontend/ # Next.js 14 App ■ ■■■ app/ ■ ■ ■■■ (applicant)/ ■ ■ ■ ■■■ session/[id]/ # Video call UI ■ ■ ■ ■■■ complete/ # Offer accepted screen ■ ■ ■■■ (dashboard)/ ■ ■ ■■■ sessions/ # M10 sessions list ■ ■ ■■■ sessions/[id]/ # M10 session detail ■ ■ ■■■ analytics/ # M10 funnel charts ■ ■■■ components/ ■ ■ ■■■ video/ # WebRTC, frame capture ■ ■ ■■■ transcript/ # Live transcript display ■ ■ ■■■ offer/ # Loan offer + EMI table ■ ■ ■■■ dashboard/ # M10 UI components ■ ■■■ lib/ ■ ■■■ webrtc.ts # M01 WebRTC helpers ■ ■■■ stt-ws.ts # M02 WebSocket STT client ■ ■■■ consent.ts # M03 phrase detection ■ ■■■ vision.ts # M04 frame capture + POST ■ ■■■ supabase.ts # Supabase client ■ ■■■ backend/ # FastAPI monorepo ■ ■■■ main.py # App entry, router mounts ■ ■■■ config.py # Env vars, settings ■ ■■■ db/ ■ ■ ■■■ supabase_client.py # Shared Supabase client ■ ■ ■■■ redis_client.py # Shared Redis client ■ ■ ■■■ migrations/ # SQL migration files ■ ■■■ modules/ ■ ■ ■■■ m01_session/ # Session & video (M01) ■ ■ ■■■ m02_stt/ # Whisper STT (M02) ■ ■ ■■■ m03_consent/ # Consent capture (M03) ■ ■ ■■■ m04_vision/ # Liveness + age (M04) ■ ■ ■■■ m05_llm/ # LLaMA3 analysis (M05) ■ ■ ■■■ m06_fraud/ # Fraud engine (M06) ■ ■ ■■■ m07_bureau/ # Eligibility engine (M07) ■ ■ ■■■ m08_offer/ # Offer + negotiate (M08) ■ ■ ■■■ m09_orchestrator/ # FSM + event bus (M09) ■ ■ ■■■ m10_compliance/ # Dashboard API (M10) ■ ■■■ shared/ ■ ■■■ auth.py # Token validation ■ ■■■ events.py # Redis event publisher ■ ■■■ models.py # Shared Pydantic models ■ ■■■ bureau-mock/ # M07 standalone microservice ■ ■■■ main.py # FastAPI on port 8001 ■ ■■■ data/bureau_seed.json # Seeded test applicant data ■ ■■■ supabase/ ■ ■■■ migrations/ # Numbered SQL files ■ ■■■ seed.sql # Dev seed data ■ ■■■ docker-compose.yml # All services wired together ■■■ .env.example # All required env vars ■■■ README.md
```

7. Environment Variables (.env)

Variable	Example Value	Used By	Notes
SUPABASE_URL	https://xxx.supabase.co	All modules	From Supabase project settings
SUPABASE_ANON_KEY	eyJ...	Frontend	Public anon key
SUPABASE_SERVICE_KEY	eyJ...	Backend	Service role key — never expose to browser
REDIS_URL	redis://localhost:6379	All backend	
GROQ_API_KEY	gsk_...	M02, M05, M08	Free at console.groq.com
SESSION_JWT_SECRET	32-char random string	M01, M09	Used to sign session tokens
GEOIP_DB_PATH	/app/data/GeoLite2-City.mmdb	M06	MaxMind GeoLite2 (free account)
BUREAU_MOCK_URL	http://localhost:8001	M07	Override with live URL in prod
WHISPER_MODEL	base.en	M02	Options: tiny.en, base.en, small.en
WHISPER_COMPUTE_TYPE	int8	M02	int8 for CPU, float16 for GPU
FRAUD_SCORE_THRESHOLD	60	M06, M07	Scores above this = high risk
AGE_DELTA_THRESHOLD	8	M04	Max allowed age delta (years)
LIVENESS_THRESHOLD	0.6	M04, M07	Min avg liveness score for PASS
NEXT_PUBLIC_API_URL	http://localhost:8000	Frontend	Backend base URL
NEXT_PUBLIC_WS_URL	ws://localhost:8000	Frontend	WebSocket base URL

8. 25-Hour Sprint Plan

Hour	Phase	Tasks	Who
0–1	Setup	Repo init, docker-compose, Supabase project, env files, run migrations	All
1–4	Foundation	M01: session tokens, WebRTC signalling, device capture M07: bureau mock microservice online	Dev A + E
1–5	Audio pipeline	M02: Whisper integration, WS audio stream, field extraction LLM call M03: consent phrase detection	Dev B
1–5	Vision	M04: MediaPipe liveness, DeepFace age, frame capture frontend	Dev C
3–7	Intelligence	M05: LLM intent + persona + CoT M06: fraud signals 1–5, composite score	Dev D + C
5–8	Eligibility	M07: rules engine wired to bureau mock M08: offer formula + EMI table	Dev E
7–11	Orchestration	M09: FSM implementation, Redis event bus, all 6 failure modes	Dev A + E
8–12	Negotiation + UI	M08: negotiation agent Frontend: full applicant flow video call UI	Dev D + B
11–16	Integration	Wire all modules through orchestrator, end-to-end happy path test	All
16–19	Dashboard	M10: sessions list, session detail, audit export endpoint	Dev B
19–22	Polish & Fix	Bug fixes, edge cases, failure mode testing, UI polish	All
22–24	Demo prep	Seed 3 demo personas (good, fraudulent, borderline), ngrok setup	All
24–25	Buffer	Rehearsal, slide deck, fallback demo video recording	All

Three Demo Personas to seed:

Persona	Profile	Expected outcome	Tests
Priya Sharma	Salaried, ■60k/month, CIBIL 740, genuine home loan	PASS → ■12L offer	Happy path
Rohit X	VPN detected, age mismatch +10yr, contradictory income	REJECT (fraud_score 82)	FM-05 fraud escalation
Meera Patil	Self-employed, ■35k, CIBIL 680, 1 DPD 30 event	REFER → human queue	R04 rule trigger

9. SQL DDL — Copy-Paste Migration

Run in Supabase SQL Editor or as a migration file. Execute in order.

```
-- 001_create_sessions.sql CREATE EXTENSION IF NOT EXISTS "pgcrypto"; CREATE TABLE sessions ( id UUID PRIMARY KEY
DEFAULT gen_random_uuid(), token_hash TEXT NOT NULL UNIQUE, phone TEXT NOT NULL, channel TEXT NOT NULL DEFAULT 'sms',
status TEXT NOT NULL DEFAULT 'initiated', ip_address INET, geo_lat DECIMAL(9,6), geo_lng DECIMAL(9,6),
device_fingerprint JSONB, device_flags JSONB DEFAULT '{}', created_at TIMESTAMPTZ NOT NULL DEFAULT now(), updated_at
TIMESTAMPTZ NOT NULL DEFAULT now(), expires_at TIMESTAMPTZ NOT NULL, completed_at TIMESTAMPTZ ); CREATE INDEX
idx_sessions_token ON sessions(token_hash); CREATE INDEX idx_sessions_status ON sessions(status); CREATE TABLE
transcripts ( id UUID PRIMARY KEY DEFAULT gen_random_uuid(), session_id UUID NOT NULL REFERENCES sessions(id) ON DELETE
CASCADE, segment_index INTEGER NOT NULL, text TEXT NOT NULL, start_ms INTEGER NOT NULL, end_ms INTEGER NOT NULL,
confidence DECIMAL(4,3), speaker TEXT DEFAULT 'applicant', created_at TIMESTAMPTZ NOT NULL DEFAULT now() ); CREATE
INDEX idx_transcripts_session ON transcripts(session_id, segment_index); CREATE TABLE extracted_fields ( id UUID
PRIMARY KEY DEFAULT gen_random_uuid(), session_id UUID NOT NULL REFERENCES sessions(id), version INTEGER NOT NULL
DEFAULT 1, name TEXT, age INTEGER, employment_type TEXT, monthly_income DECIMAL(12,2), loan_purpose TEXT,
requested_amount DECIMAL(12,2), confidence_map JSONB NOT NULL, low_confidence_fields TEXT[], created_at TIMESTAMPTZ
NOT NULL DEFAULT now() ); CREATE TABLE consent_records ( id UUID PRIMARY KEY DEFAULT gen_random_uuid(), session_id UUID
NOT NULL REFERENCES sessions(id), consent_type TEXT NOT NULL, phrase_detected TEXT NOT NULL, segment_text TEXT NOT
NULL, segment_hash TEXT NOT NULL, timestamp_ms BIGINT NOT NULL, detected_at TIMESTAMPTZ NOT NULL DEFAULT now() ); --
Append-only policy ALTER TABLE consent_records ENABLE ROW LEVEL SECURITY; CREATE POLICY no_update_consent ON
consent_records FOR UPDATE USING (false); CREATE POLICY no_delete_consent ON consent_records FOR DELETE USING (false);
CREATE TABLE vision_events ( id UUID PRIMARY KEY DEFAULT gen_random_uuid(), session_id UUID NOT NULL REFERENCES
sessions(id), frame_index INTEGER NOT NULL, captured_at_ms BIGINT NOT NULL, liveness_score DECIMAL(4,3) NOT NULL,
age_estimate INTEGER, blink_detected BOOLEAN DEFAULT false, head_pose JSONB, flags TEXT[], model_used TEXT NOT NULL,
created_at TIMESTAMPTZ NOT NULL DEFAULT now() ); CREATE INDEX idx_vision_session ON vision_events(session_id,
frame_index); CREATE TABLE llm_assessments ( id UUID PRIMARY KEY DEFAULT gen_random_uuid(), session_id UUID NOT NULL
REFERENCES sessions(id), assessment_version INTEGER DEFAULT 1, intent_category TEXT NOT NULL, intent_confidence
DECIMAL(4,3) NOT NULL, risk_persona TEXT NOT NULL, persona_confidence DECIMAL(4,3) NOT NULL, cot_trace JSONB NOT NULL,
tone_flags TEXT[], contradiction_detected BOOLEAN DEFAULT false, contradiction_detail TEXT, full_prompt TEXT NOT NULL,
model_used TEXT NOT NULL, tokens_used INTEGER, latency_ms INTEGER, created_at TIMESTAMPTZ NOT NULL DEFAULT now() );
CREATE TABLE fraud_reports ( id UUID PRIMARY KEY DEFAULT gen_random_uuid(), session_id UUID NOT NULL UNIQUE REFERENCES
sessions(id), fraud_score INTEGER NOT NULL CHECK (fraud_score BETWEEN 0 AND 100), risk_level TEXT NOT NULL,
geo_mismatch_score INTEGER NOT NULL, device_anomaly_score INTEGER NOT NULL, age_inconsistency_score INTEGER NOT NULL,
transcript_contradiction_score INTEGER NOT NULL, velocity_score INTEGER NOT NULL, signal_breakdown JSONB NOT NULL,
created_at TIMESTAMPTZ NOT NULL DEFAULT now() ); CREATE TABLE bureau_responses ( id UUID PRIMARY KEY DEFAULT
gen_random_uuid(), session_id UUID NOT NULL REFERENCES sessions(id), applicant_id TEXT NOT NULL, credit_score INTEGER
NOT NULL, active_loan_count INTEGER NOT NULL, dpd_history JSONB NOT NULL, written_off_accounts INTEGER NOT NULL DEFAULT
0, inquiry_count_90d INTEGER NOT NULL, foir DECIMAL(5,4) NOT NULL, eligibility_decision TEXT NOT NULL, rules_evaluated
JSONB NOT NULL, response_source TEXT DEFAULT 'mock', fetched_at TIMESTAMPTZ NOT NULL DEFAULT now() ); CREATE TABLE
loan_offers ( id UUID PRIMARY KEY DEFAULT gen_random_uuid(), session_id UUID NOT NULL REFERENCES sessions(id),
offer_version INTEGER DEFAULT 1, principal_amount DECIMAL(12,2) NOT NULL, interest_rate_pct DECIMAL(5,2) NOT NULL,
tenure_months INTEGER, emi_amount DECIMAL(10,2), emi_table JSONB NOT NULL, negotiation_round INTEGER NOT NULL DEFAULT
0, status TEXT NOT NULL DEFAULT 'pending', accepted_at TIMESTAMPTZ, created_at TIMESTAMPTZ NOT NULL DEFAULT now() );
CREATE TABLE audit_events ( id UUID PRIMARY KEY DEFAULT gen_random_uuid(), session_id UUID NOT NULL REFERENCES
sessions(id), event_type TEXT NOT NULL, from_stage TEXT, to_stage TEXT, module TEXT NOT NULL, payload JSONB NOT NULL,
actor TEXT DEFAULT 'system', created_at TIMESTAMPTZ NOT NULL DEFAULT now() ); CREATE INDEX idx_audit_session ON
audit_events(session_id, created_at); ALTER TABLE audit_events ENABLE ROW LEVEL SECURITY; CREATE POLICY
no_update_audit ON audit_events FOR UPDATE USING (false); CREATE POLICY no_delete_audit ON audit_events FOR DELETE
USING (false); -- auto-update updated_at on sessions CREATE OR REPLACE FUNCTION update_updated_at() RETURNS TRIGGER AS
$$ BEGIN NEW.updated_at = now(); RETURN NEW; END; $$ LANGUAGE plpgsql; CREATE TRIGGER trg_sessions_updated_at BEFORE
UPDATE ON sessions FOR EACH ROW EXECUTE FUNCTION update_updated_at();
```

10. docker-compose.yml Skeleton

```
version: '3.9' services: frontend: build: ./frontend ports: ["3000:3000"] environment: -
NEXT_PUBLIC_API_URL=http://localhost:8000 - NEXT_PUBLIC_WS_URL=ws://localhost:8000 -
NEXT_PUBLIC_SUPABASE_URL=${SUPABASE_URL} - NEXT_PUBLIC_SUPABASE_ANON_KEY=${SUPABASE_ANON_KEY} depends_on:
[backend] backend: build: ./backend ports: ["8000:8000"] environment: - SUPABASE_URL=${SUPABASE_URL} -
SUPABASE_SERVICE_KEY=${SUPABASE_SERVICE_KEY} - REDIS_URL=redis://redis:6379 - GROQ_API_KEY=${GROQ_API_KEY} -
SESSION_JWT_SECRET=${SESSION_JWT_SECRET} - BUREAU_MOCK_URL=http://bureau-mock:8001 -
GEOIP_DB_PATH=/app/data/GeoLite2-City.mmdb - WHISPER_MODEL=base.en - WHISPER_COMPUTE_TYPE=int8 -
FRAUD_SCORE_THRESHOLD=60 volumes: - ./data:/app/data # GeoLite2 DB, Whisper model cache depends_on: [redis]
bureau-mock: build: ./bureau-mock ports: ["8001:8001"] redis: image: redis:7-alpine ports: ["6379:6379"]
command: redis-server --maxmemory 256mb --maxmemory-policy allkeys-lru volumes: redis_data:
```

Quick start commands:

```
# 1. Clone and configure cp .env.example .env # fill in SUPABASE_URL, GROQ_API_KEY, etc. # 2. Download
GeoLite2 (free MaxMind account required) mkdir -p data && mv GeoLite2-City.mmdb data/ # 3. Start all
services docker compose up --build # 4. Run Supabase migrations python backend/db/run_migrations.py #
5. Seed demo personas python backend/db/seed_demo.py # 6. Expose for video-call demo ngrok http 3000
```

11. Pre-Demo Checklist

#	Check	Verified by	Status
1	All 3 demo personas seeded in Supabase + bureau mock	Dev E	[ ]
2	ngrok tunnel running and URL shared with judges	Dev A	[ ]
3	Happy path (Priya): full flow in <8 min end-to-end	Dev D	[ ]
4	Fraud path (Rohit): session flagged, audit event visible in dashboard	Dev C	[ ]
5	REFER path (Meera): REFER decision, visible in queue	Dev E	[ ]
6	Consent export artifact downloadable from dashboard	Dev B	[ ]
7	Regulatory audit report generated live (not pre-baked)	Dev B	[ ]
8	Redis velocity check demo: same device triggers flag	Dev C	[ ]
9	FM-04 video reconnect demo: disconnect and resume session	Dev A	[ ]
10	Audit event log replay: full session reconstructable from audit_events	Dev B	[ ]
11	Liveness demo: hold up photo to camera → flagged	Dev C	[ ]
12	Fallback demo video recorded (in case live fails)	All	[ ]

Good luck, team. Build one thing at a time. Get the happy path working first. Everything else is polish. The judges care most about: (1) working demo, (2) clear explainability, (3) genuine audit trail. Tier 9 (orchestrator + fallbacks) and Tier 10 (compliance dashboard) are your differentiators — most teams will skip them.